



UNIVERSITI MALAYSIA TERENGGANU

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONORS

SEMESTER II 2025/2026

CSA3023

WEB-BASED APPLICATION DEVELOPMENT

LAB 5:

JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)

Prepared by:

NURSYAHEERA BINTI MOHD HISHAM

Prepared for:

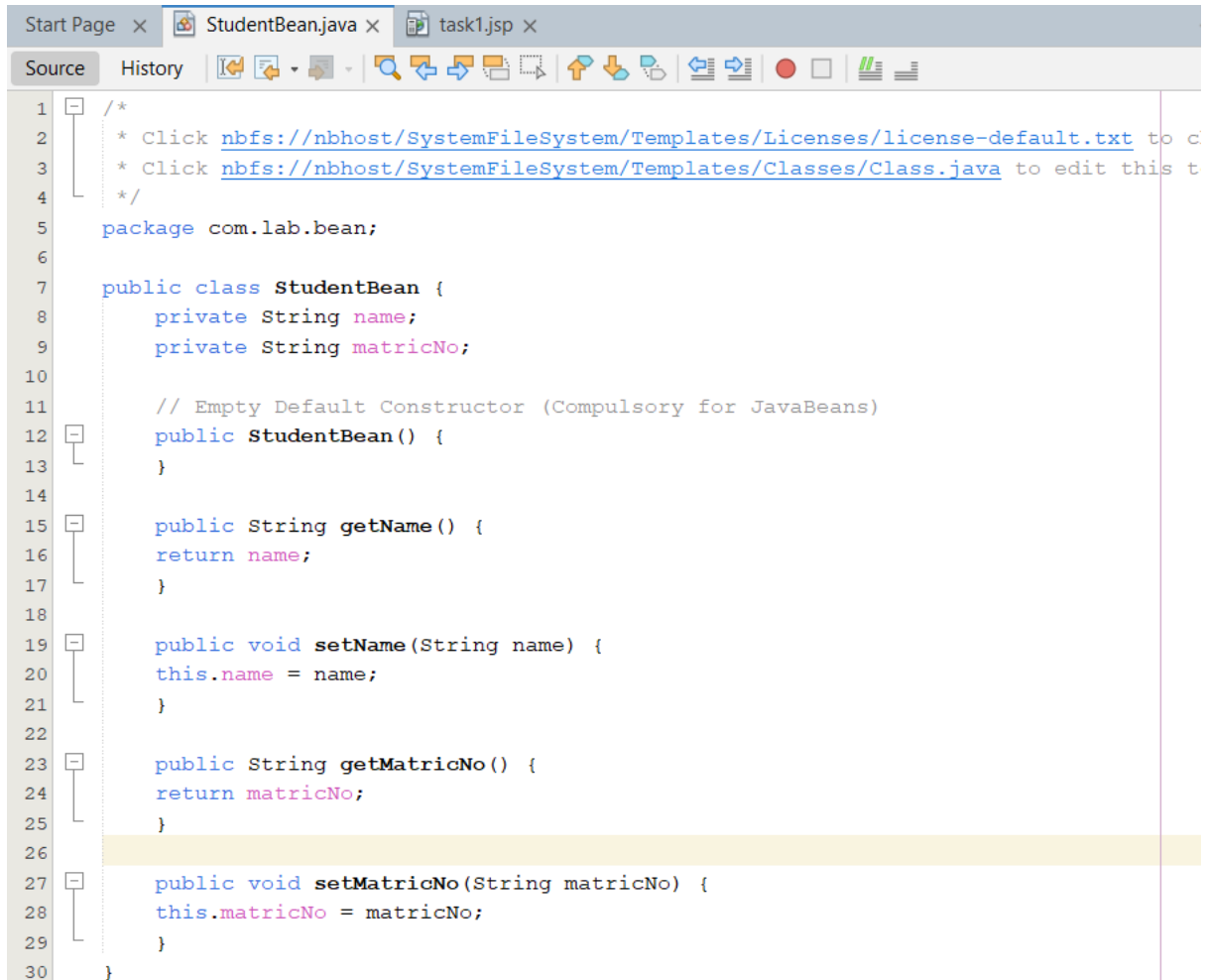
SIR. MOHAMAD ARIZAL SHAMSIL MAT RIFFIN

Date:

12 MAY 2026

Task 1: Using Scriptlet to Access a Simple JavaBeans

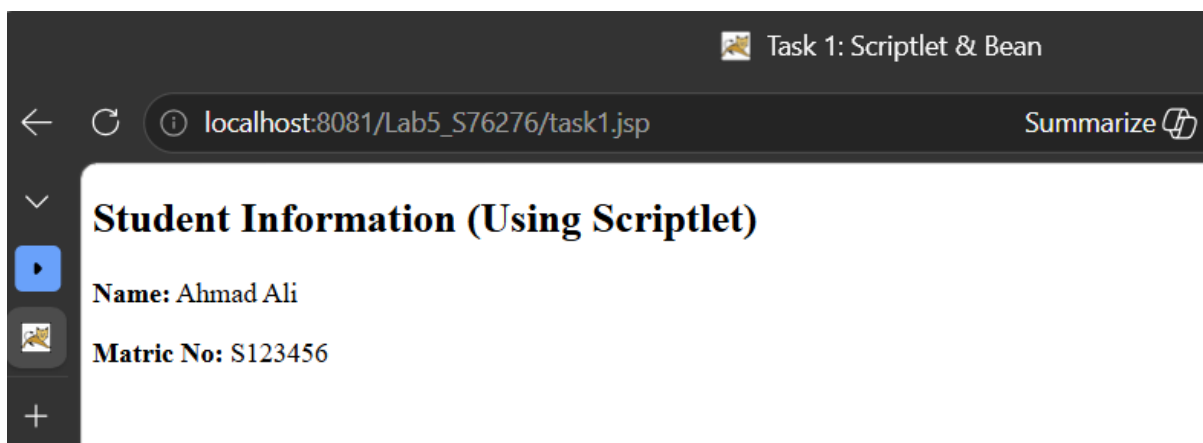
Objective: To understand how to create a basic JavaBean and instantiate it inside a JSP page using standard Java scriptlets.



```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to c
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this t
4   */
5   package com.lab.bean;
6
7   public class StudentBean {
8       private String name;
9       private String matricNo;
10
11       // Empty Default Constructor (Compulsory for JavaBeans)
12       public StudentBean() {
13       }
14
15       public String getName() {
16       return name;
17       }
18
19       public void setName(String name) {
20       this.name = name;
21       }
22
23       public String getMatricNo() {
24       return matricNo;
25       }
26
27       public void setMatricNo(String matricNo) {
28       this.matricNo = matricNo;
29       }
30   }
```

```
Start Page x StudentBean.java x task1.jsp x
Source History
1 <%--
2     Document    : task1
3     Created on  : 12 May 2026, 4:57:06 pm
4     Author     : nursyaheera binti mohd hisham
5 --%>
6
7 <%@page import="com.lab.bean.StudentBean"%>
8 <%@page contentType="text/html" pageEncoding="UTF-8"%>
9 <!DOCTYPE html>
10 <html>
11     <head>
12         <title>Task 1: Scriptlet & Bean</title>
13     </head>
14     <body>
15         <h2>Student Information (Using Scriptlet)</h2>
16
17         <%
18             // Instantiating the bean
19             StudentBean student = new StudentBean();
20             student.setName("Ahmad Ali");
21             student.setMatricNo("S123456");
22         %>
23
24         <p><strong>Name:</strong> <%= student.getName() %></p>
25         <p><strong>Matric No:</strong> <%= student.getMatricNo() %></p>
26     </body>
27 </html>
```

Output:



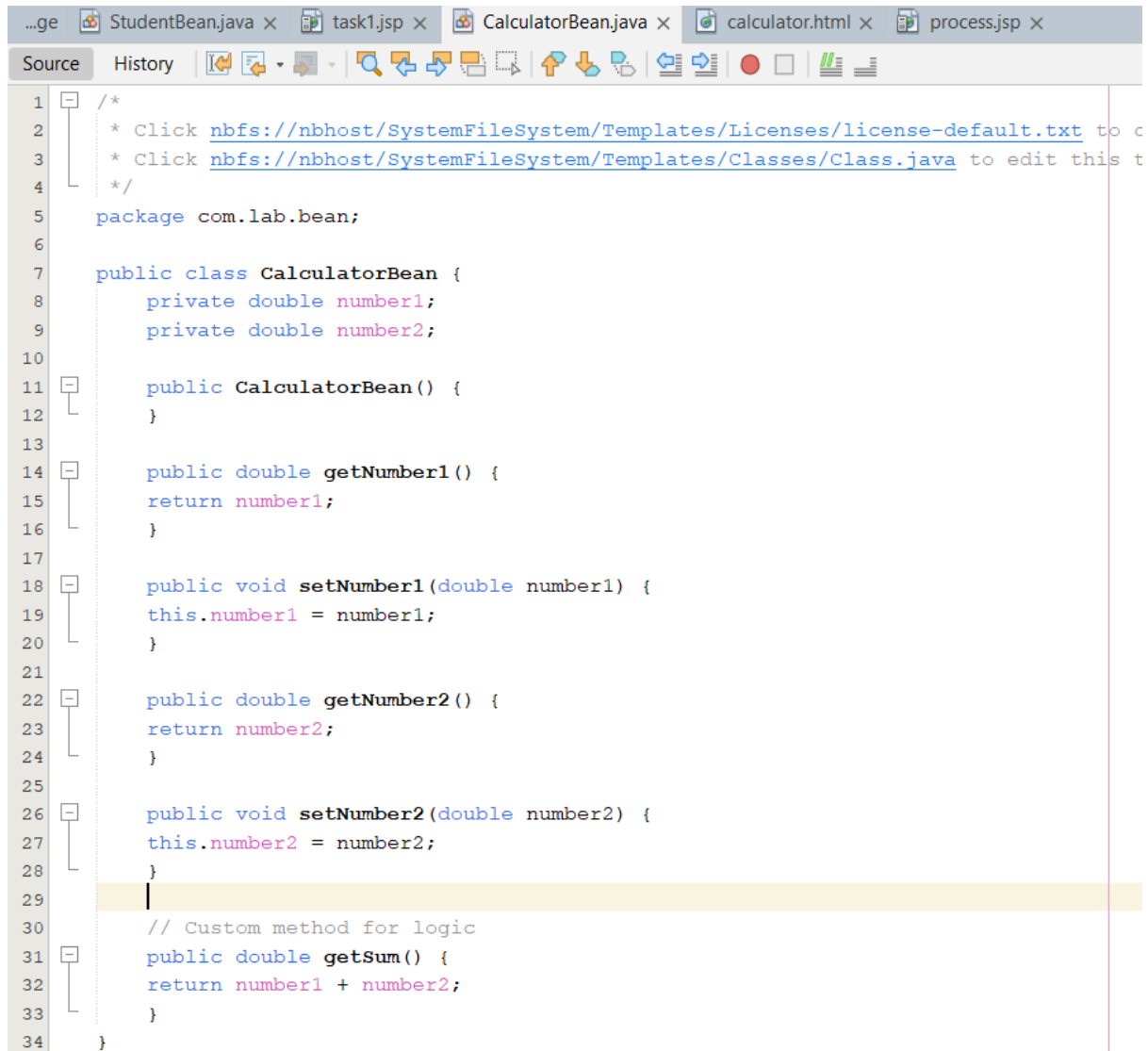
Reflections

1. What is the main disadvantage of mixing Java scriptlets directly inside HTML codes?
Answer:
The main disadvantage is that it makes the JSP page difficult to read, maintain, and debug because Java code and HTML code are mixed together. It also reduces code reusability and violates the separation between business logic and presentation.
2. Why is it compulsory for a JavaBean class to have an empty default constructor?
Answer:

An empty default constructor is compulsory because JSP and JavaBeans use it to automatically create an object instance. Without the default constructor, the bean cannot be instantiated properly by JSP action tags such as `<jsp:useBean>`.

Task 2: Problem Solving using JavaBeans

Objective: To utilize JSP standard action tags (,) to process HTML form data automatically without using scriptlets.

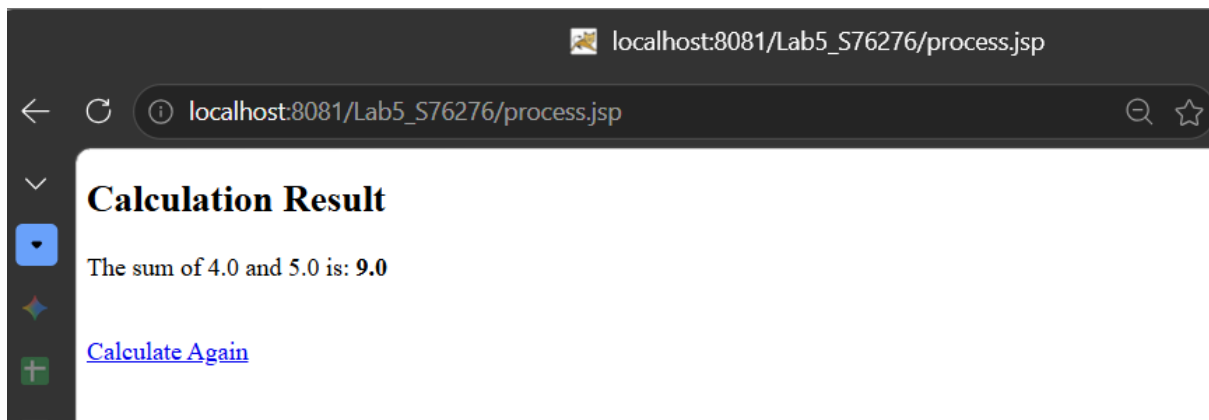
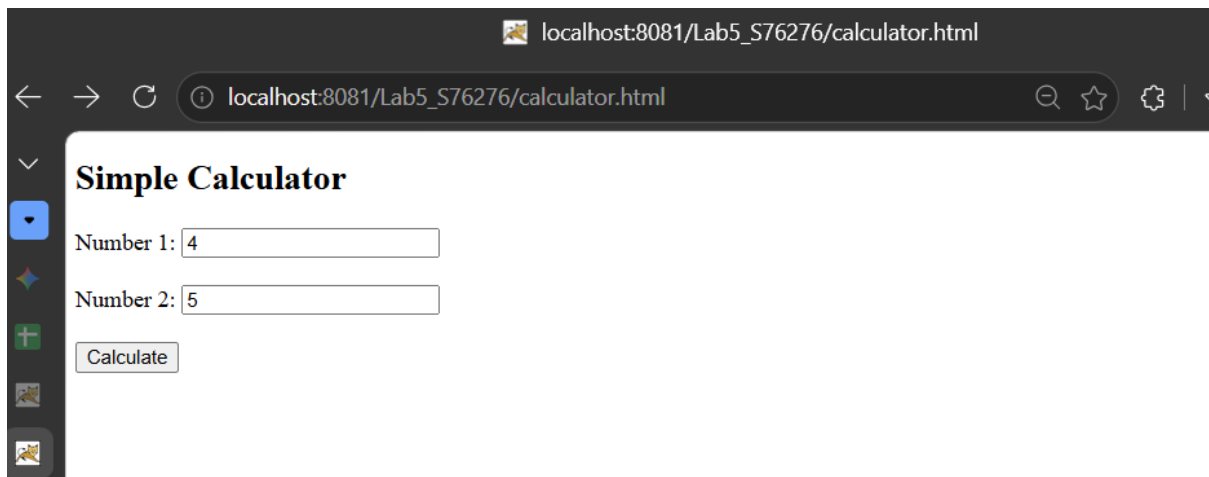


```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to c
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this t
4   */
5  package com.lab.bean;
6
7  public class CalculatorBean {
8      private double number1;
9      private double number2;
10
11     public CalculatorBean() {
12     }
13
14     public double getNumber1() {
15         return number1;
16     }
17
18     public void setNumber1(double number1) {
19         this.number1 = number1;
20     }
21
22     public double getNumber2() {
23         return number2;
24     }
25
26     public void setNumber2(double number2) {
27         this.number2 = number2;
28     }
29
30     // Custom method for logic
31     public double getSum() {
32         return number1 + number2;
33     }
34 }
```

```
...ge StudentBean.java x task1.jsp x CalculatorBean.java x calculator.html x process.jsp x
Source History
1 <!DOCTYPE html>
2 <!--
3 Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
4 Click nbfs://nbhost/SystemFileSystem/Templates/JSP\_Servlet/Html.html to edit this template
5 -->
6 <html>
7 <body>
8 <h2>Simple Calculator</h2>
9 <form action="process.jsp" method="POST">
10     Number 1: <input type="text" name="number1"><br><br>
11     Number 2: <input type="text" name="number2"><br><br>
12     <input type="submit" value="Calculate">
13 </form>
14 </body>
15 </html>
16
```

```
...ge StudentBean.java x task1.jsp x CalculatorBean.java x calculator.html x process.jsp x
Source History
1 <%--
2     Document    : process
3     Created on  : 12 May 2026, 5:09:05 pm
4     Author      : nursyaheera binti mohd hisham
5 --%>
6
7 <%@page contentType="text/html" pageEncoding="UTF-8"%>
8 <!DOCTYPE html>
9 <html>
10 <body>
11     <h2>Calculation Result</h2>
12
13     <jsp:useBean id="calc" class="com.lab.bean.CalculatorBean" />
14
15     <jsp:setProperty name="calc" property="*" />
16
17     <p>The sum of
18         <jsp:getProperty name="calc" property="number1" /> and
19         <jsp:getProperty name="calc" property="number2" /> is:
20         <strong><jsp:getProperty name="calc" property="sum" /></strong>
21     </p>
22
23     <br><a href="calculator.html">Calculate Again</a>
24 </body>
25 </html>
26
```

Output:



Reflections

1. Explain how the `property="*"` attribute works in the tag.

Answer:

The `property="*"` attribute automatically matches the HTML form input names with the JavaBean property names. If the form field name and bean variable name are the same, the values are automatically assigned to the bean without manually calling setter methods.

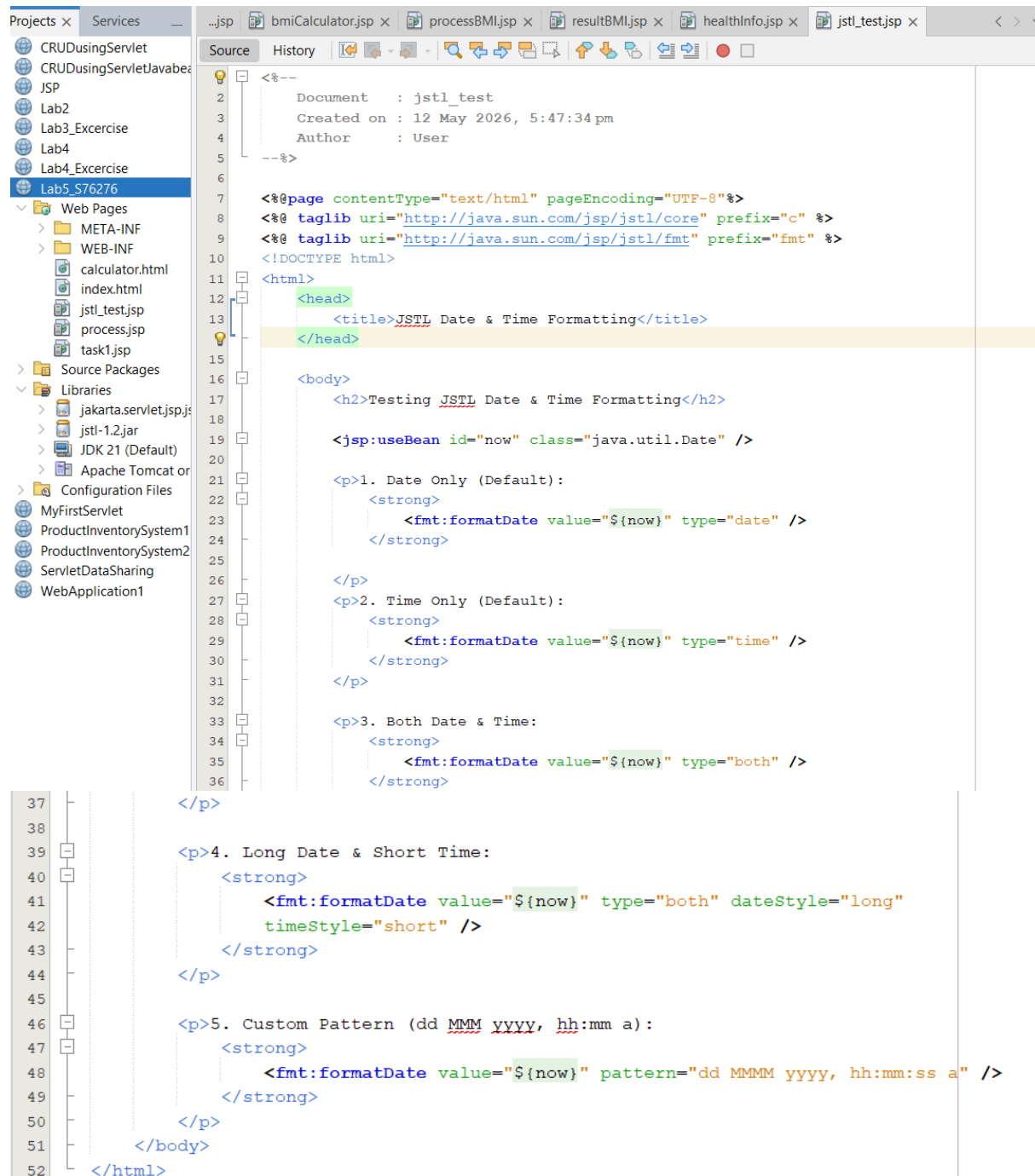
2. How do JSP action tags improve code readability compared to the scriptlets used in Task 1?

Answer:

JSP action tags improve readability because they reduce the amount of Java code written inside the JSP page. The code becomes cleaner, easier to understand, and easier to maintain since the logic is handled automatically using predefined tags.

Task 3: Installing JSTL Taglibs

Objective: To download, install, configure, and test the JavaServer Pages Standard Tag Library (JSTL) in a web project.



```
<!--
  Document   : jstl_test
  Created on : 12 May 2026, 5:47:34 pm
  Author      : User
-->

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
<!DOCTYPE html>
<html>
<head>
<title>JSTL Date & Time Formatting</title>
</head>
<body>
<h2>Testing JSTL Date & Time Formatting</h2>

<jsp:useBean id="now" class="java.util.Date" />

<p>1. Date Only (Default):
  <strong>
    <fmt:formatDate value="{now}" type="date" />
  </strong>
</p>

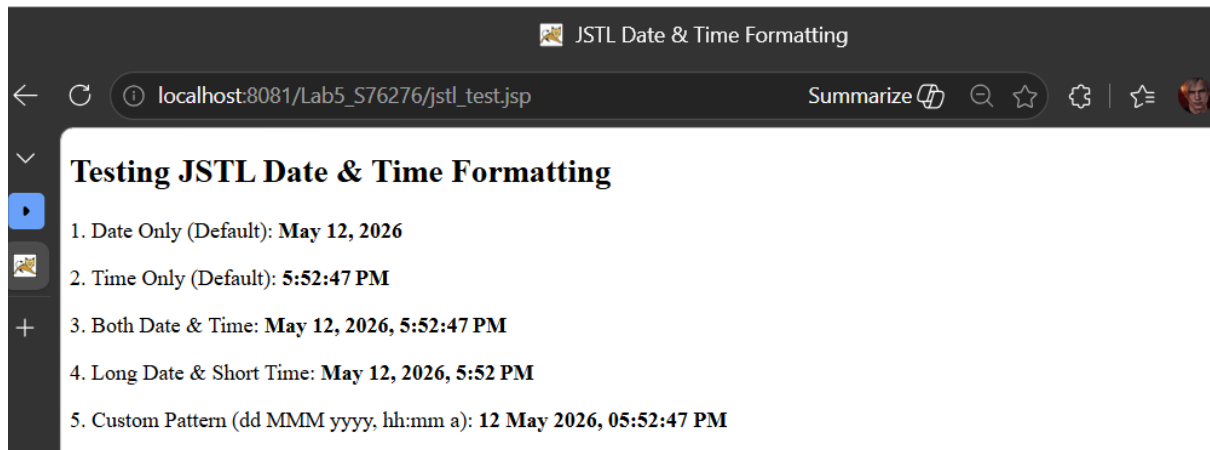
<p>2. Time Only (Default):
  <strong>
    <fmt:formatDate value="{now}" type="time" />
  </strong>
</p>

<p>3. Both Date & Time:
  <strong>
    <fmt:formatDate value="{now}" type="both" />
  </strong>
</p>

<p>4. Long Date & Short Time:
  <strong>
    <fmt:formatDate value="{now}" type="both" dateStyle="long"
      timeStyle="short" />
  </strong>
</p>

<p>5. Custom Pattern (dd MMM yyyy, hh:mm a):
  <strong>
    <fmt:formatDate value="{now}" pattern="dd MMMM yyyy, hh:mm:ss a" />
  </strong>
</p>
</body>
</html>
```

Output:

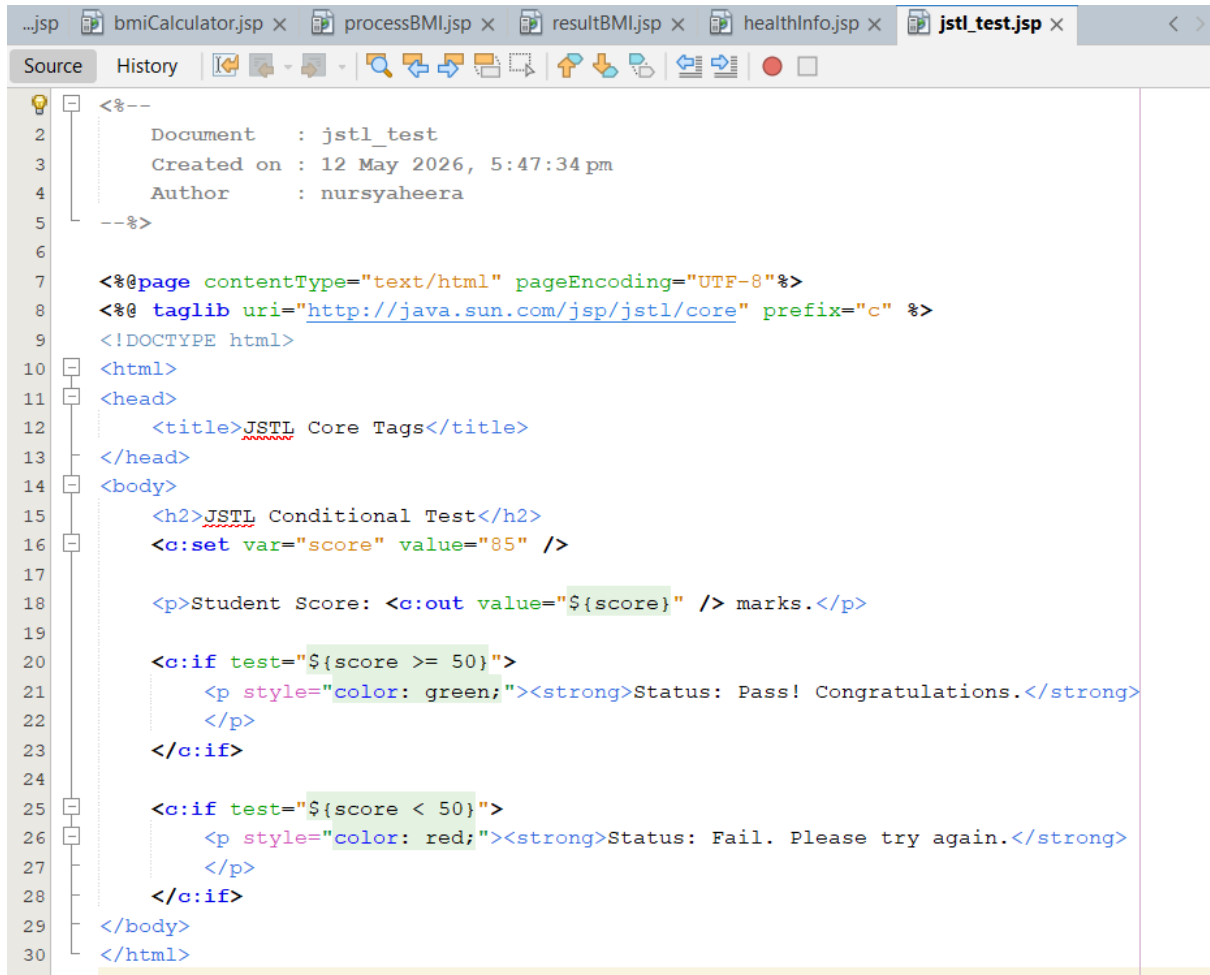


Reflections

1. What is the purpose of the uri and prefix attributes in the taglib directive?
Answer:
The uri attribute identifies the JSTL library being used, while the prefix attribute defines the shortcut name used to access JSTL tags inside the JSP page.
2. Why do we need separate taglib directives for core and fmt?
Answer:
Separate taglib directives are needed because each JSTL library provides different functions. The Core library handles logic and iteration tags, while the Format (fmt) library handles formatting for numbers, dates, currencies, and localization.

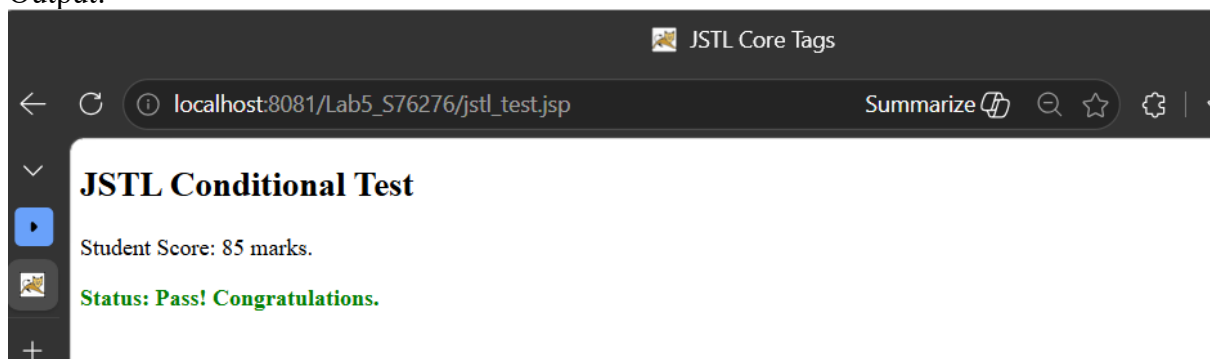
Task 4: Using Java Standard Tag Library (JSTL) Core Tags

Objective: To perform variable declarations, value output, and conditional checks using JSTL Core Tags



```
1 <!--
2   Document   : jstl_test
3   Created on : 12 May 2026, 5:47:34 pm
4   Author    : nursyaheera
5 -->
6
7 <%@page contentType="text/html" pageEncoding="UTF-8"%>
8 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
9 <!DOCTYPE html>
10 <html>
11 <head>
12   <title>JSTL Core Tags</title>
13 </head>
14 <body>
15   <h2>JSTL Conditional Test</h2>
16   <c:set var="score" value="85" />
17
18   <p>Student Score: <c:out value="${score}" /> marks.</p>
19
20   <c:if test="${score >= 50}">
21     <p style="color: green;"><strong>Status: Pass! Congratulations.</strong>
22   </p>
23   </c:if>
24
25   <c:if test="${score < 50}">
26     <p style="color: red;"><strong>Status: Fail. Please try again.</strong>
27   </p>
28   </c:if>
29 </body>
30 </html>
```

Output:



Reflections

1. How does the tag help in preventing Cross-Site Scripting (XSS) attacks?

Answer:

The `<c:out>` tag automatically escapes special HTML characters before displaying output. This prevents malicious scripts from being executed in the browser, helping to reduce XSS attacks.

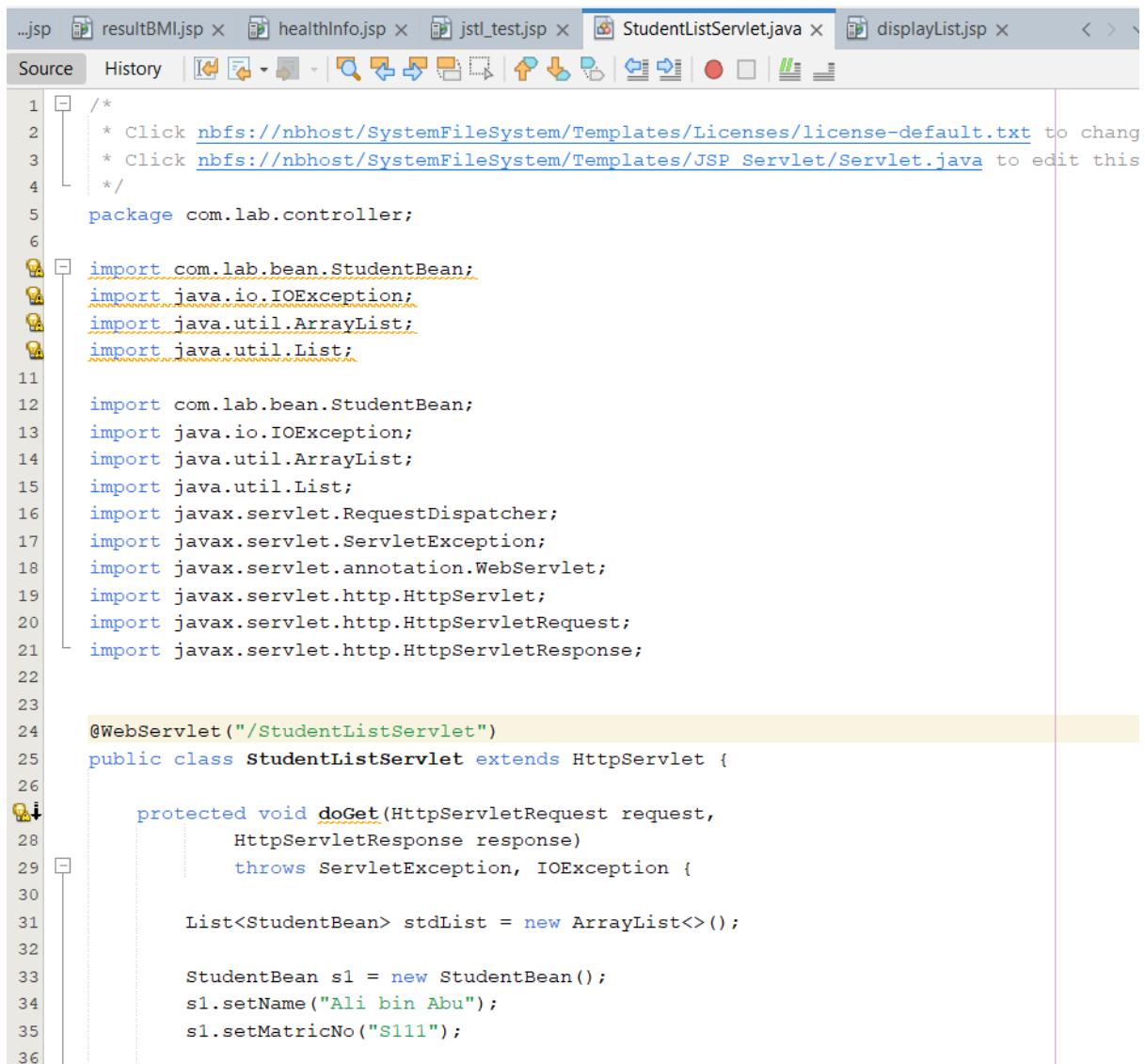
2. The tag does not have an "else" statement. Which JSTL tags should be used if you need an if-else logic structure?

Answer:

The JSTL tags `<c:choose>`, `<c:when>`, and `<c:otherwise>` should be used to create an if-else logic structure.

Task 5: Using JSP Standard Tag Library (JSTL) for Collections

Objective: To combine JavaBeans and JSTL by iterating through a collection of objects using the loop.



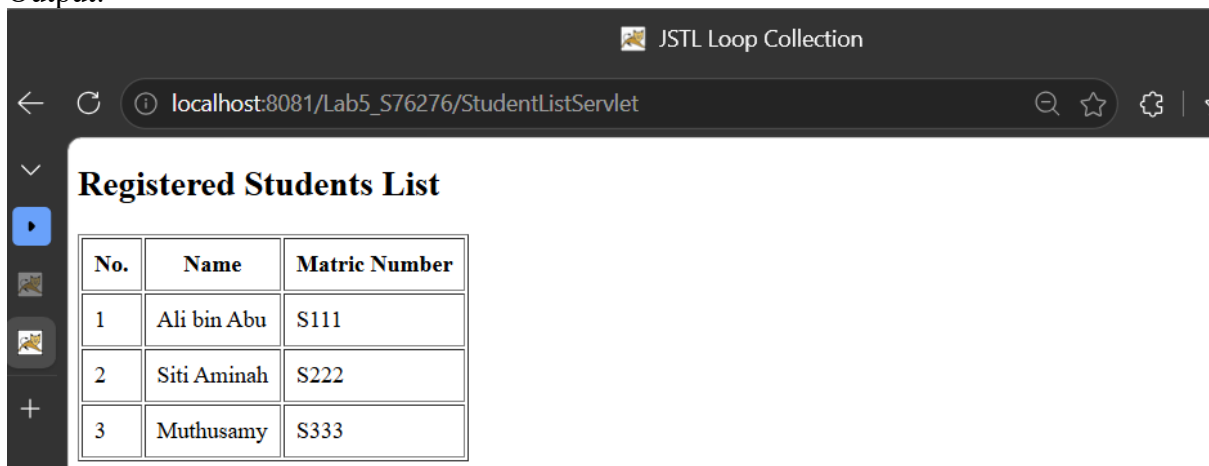
```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to chang
3   * Click nbfs://nbhost/SystemFileSystem/Templates/JSP\_Servlet/Servlet.java to edit this
4   */
5  package com.lab.controller;
6
7  import com.lab.bean.StudentBean;
8  import java.io.IOException;
9  import java.util.ArrayList;
10 import java.util.List;
11
12 import com.lab.bean.StudentBean;
13 import java.io.IOException;
14 import java.util.ArrayList;
15 import java.util.List;
16 import javax.servlet.RequestDispatcher;
17 import javax.servlet.ServletException;
18 import javax.servlet.annotation.WebServlet;
19 import javax.servlet.http.HttpServlet;
20 import javax.servlet.http.HttpServletRequest;
21 import javax.servlet.http.HttpServletResponse;
22
23
24 @WebServlet("/StudentListServlet")
25 public class StudentListServlet extends HttpServlet {
26
27     protected void doGet(HttpServletRequest request,
28                          HttpServletResponse response)
29         throws ServletException, IOException {
30
31         List<StudentBean> stdList = new ArrayList<>();
32
33         StudentBean s1 = new StudentBean();
34         s1.setName("Ali bin Abu");
35         s1.setMatricNo("S111");
36     }
}
```

```

37     StudentBean s2 = new StudentBean();
38     s2.setName("Siti Aminah");
39     s2.setMatricNo("S222");
40
41     StudentBean s3 = new StudentBean();
42     s3.setName("Muthusamy");
43     s3.setMatricNo("S333");
44
45     stdList.add(s1);
46     stdList.add(s2);
47     stdList.add(s3);
48
49     request.setAttribute("listData", stdList);
50
51     RequestDispatcher rd =
52         request.getRequestDispatcher("displayList.jsp");
53
54     rd.forward(request, response);
55 }
56

```

Output:



No.	Name	Matric Number
1	Ali bin Abu	S111
2	Siti Aminah	S222
3	Muthusamy	S333

Reflections

1. Describe how the items and var attributes work together in the tag.

Answer:

The items attribute represents the collection or list to be looped through, while the var attribute stores each individual object during every iteration of the loop.

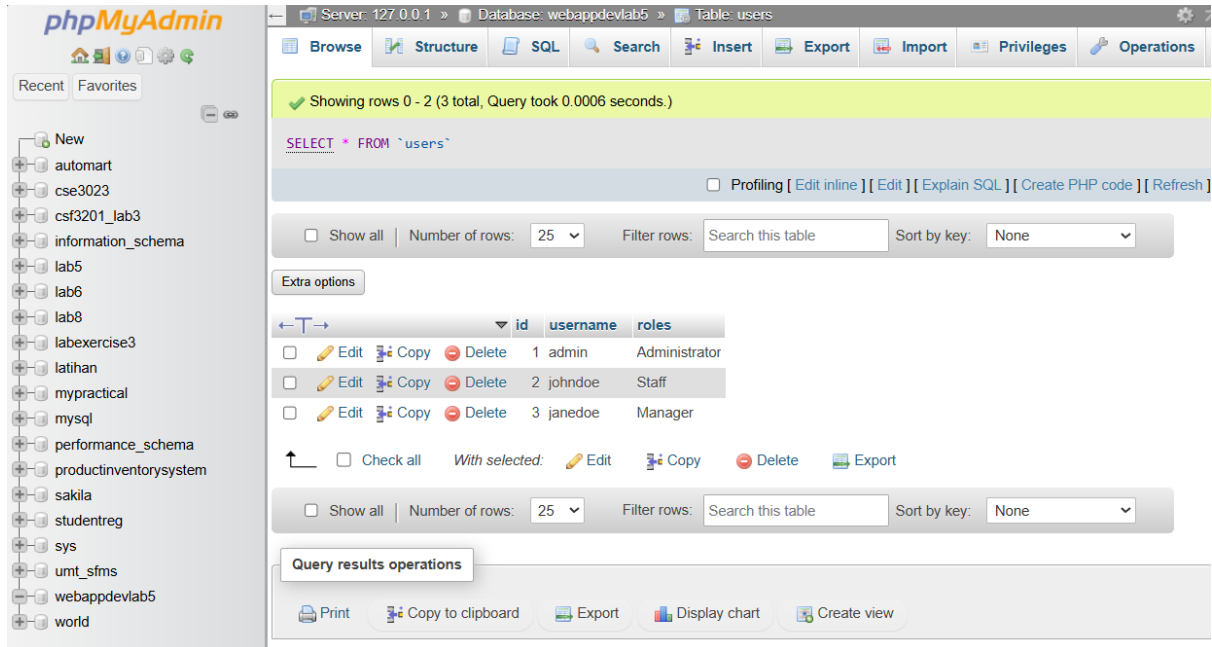
2. Why is it important to run the Servlet instead of the JSP file directly in this task?

Answer:

It is important because the Servlet prepares and sends the data (listData) to the JSP page. If the JSP file is run directly, the required data will not exist, causing empty output or errors.

Task 6: Database Access using JSTL SQL Tags

Objective: To connect to a MySQL database and execute SQL queries directly from a JSP page using the JSTL SQL Tag Library.

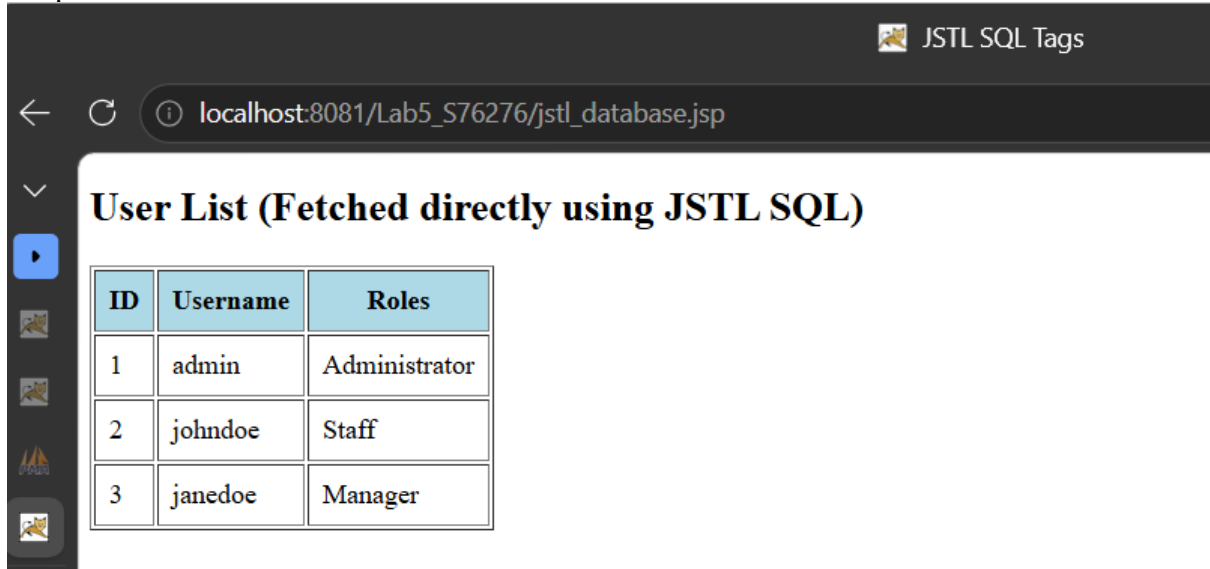


The screenshot displays the phpMyAdmin web interface. On the left is a sidebar with a tree view of databases and tables. The main panel shows the 'users' table selected within the 'webappdevlab5' database. A green status bar at the top indicates 'Showing rows 0 - 2 (3 total, Query took 0.0006 seconds.)'. Below this, the SQL query 'SELECT * FROM `users`' is entered. A toolbar with icons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, and Operations is visible. Below the toolbar, there are controls for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (a search box), and 'Sort by key' (set to None). An 'Extra options' button is also present. The table data is displayed in a grid with columns 'id', 'username', and 'roles'. Each row has checkboxes and icons for Edit, Copy, and Delete. At the bottom, there are buttons for 'Check all', 'With selected', 'Edit', 'Copy', 'Delete', and 'Export'. A 'Query results operations' section at the very bottom includes buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

	id	username	roles
☐	1	admin	Administrator
☐	2	johndoe	Staff
☐	3	janedoe	Manager

```
...jsp healthInfo.jsp x jstl_test.jsp x StudentListServlet.java x displayList.jsp x jstl_database.jsp x
Source History
1 <!--
2     Document    : jstl_database
3     Created on  : 12 May 2026, 6:19:59 pm
4     Author     : nursyaheera
5 -->
6
7 <%@page contentType="text/html" pageEncoding="UTF-8"%>
8 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
9 <%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
10 <!DOCTYPE html>
11 <html>
12 <head>
13     <title>JSTL SQL Tags</title>
14 </head>
15 <body>
16     <h2>User List (Fetched directly using JSTL SQL)</h2>
17     <sql:setDataSource var="dbConnection" driver="com.mysql.cj.jdbc.Driver"
18         url="jdbc:mysql://localhost:3306/webappdevlab5"
19         user="root" password="admin"/>
20
21     <sql:query dataSource="${dbConnection}" var="result">
22         SELECT * FROM users;
23     </sql:query>
24     <table border="1" cellpadding="8">
25         <thead>
26             <tr style="background-color: lightblue;">
27                 <th>ID</th>
28                 <th>Username</th>
29                 <th>Roles</th>
30             </tr>
31         </thead>
32         <tbody>
33             <c:forEach var="row" items="${result.rows}">
34                 <tr>
35                     <td><c:out value="${row.id}" /></td>
36                     <td><c:out value="${row.username}" /></td>
37                     <td><c:out value="${row.roles}" /></td>
38                 </tr>
39             </c:forEach>
40         </tbody>
41     </table>
42 </body>
43 </html>
44
45
```

Output:



The screenshot shows a web browser window with the address bar displaying `localhost:8081/Lab5_S76276/jstl_database.jsp`. The page title is "JSTL SQL Tags". Below the title, the heading "User List (Fetched directly using JSTL SQL)" is displayed. A table with three columns (ID, Username, Roles) and three rows of user data is shown.

ID	Username	Roles
1	admin	Administrator
2	johndoe	Staff
3	janedoe	Manager

Reflections

- The JSTL tag allows you to retrieve data without writing Servlets or DAO classes. Why is this considered a bad practice (violating the MVC pattern) for large-scale enterprise applications?

Answer:

This is considered bad practice because it mixes database logic directly inside the presentation layer (JSP). It violates the MVC architecture where database operations should belong to the Model layer. In large applications, this approach makes the code difficult to maintain, reuse, secure, and debug. Using Servlets and DAO classes provides better organization, scalability, and separation of responsibilities.

Exercise: Employee Payroll Display System

Based on the concepts you have learned in this lab (JavaBeans, Servlets as Controllers, and JSTL for the View), you are required to independently develop a simple Employee Payroll Display System.

Please complete this exercise by fulfilling the following requirements:

1. **JavaBean (Model):** Create a class named `Employee` in the `com.lab.bean` package with the following private attributes: `empId` (String), `name` (String), `department` (String), and `basicSalary` (double). Include the standard empty constructor and the getter/setter methods for all attributes.
2. **Servlet (Controller):** Create a `PayrollServlet` in the `com.lab.controller` package. Inside this Servlet's `doGet()` method:
 - Create a list (`ArrayList`) and populate it with at least 5 different `Employee` objects.
 - Share this list to the JSP using `request.setAttribute("employeeList", list);`.
 - Forward the request to a view file named `payroll_view.jsp`.
3. **JSP & JSTL (View):** Create `payroll_view.jsp` to display the data:
 - Import the JSTL core taglib directive at the very top of the file.
 - Use the `<forEach>` tag to iterate through the `employeeList` and display their details in a well-formatted HTML table.
 - **Logic Challenge:** Use the `<if>`, `<choose>`, and `<when>` tags inside the table to add a new "Status" column. If the employee's basic salary is RM3000 or above, display the text "Senior"; otherwise, display "Junior".

...va displayList.jsp x jstl_database.jsp x Employee.java x PayrollServlet.java x payroll_view.jsp x

Source History

```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change the
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package com.lab.bean;
6
7   public class Employee {
8
9       private String empId;
10      private String name;
11      private String department;
12      private double basicSalary;
13
14      // Empty Constructor
15      public Employee() {
16
17      }
18
19      // Getter and Setter Methods
20
21      public String getEmpId() {
22          return empId;
23      }
24
25      public void setEmpId(String empId) {
26          this.empId = empId;
27      }
28
29      public String getName() {
30          return name;
31      }
32
33      public void setName(String name) {
34          this.name = name;
35      }
36
37      public String getDepartment() {
38          return department;
39      }
40
41      public void setDepartment(String department) {
42          this.department = department;
43      }
44
45      public double getBasicSalary() {
46          return basicSalary;
47      }
48
49      public void setBasicSalary(double basicSalary) {
50          this.basicSalary = basicSalary;
51      }
52  }
```


...va displayList.jsp x jstl_database.jsp x Employee.java x PayrollServlet.java x payroll_view.jsp x

Source History

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change thi
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package com.lab.controller;
6
7  import com.lab.bean.Employee;
8
9  import java.io.IOException;
10 import java.util.ArrayList;
11 import java.util.List;
12
13 import javax.servlet.RequestDispatcher;
14 import javax.servlet.ServletException;
15 import javax.servlet.annotation.WebServlet;
16 import javax.servlet.http.HttpServlet;
17 import javax.servlet.http.HttpServletRequest;
18 import javax.servlet.http.HttpServletResponse;
19
20 @WebServlet("/PayrollServlet")
21
22 public class PayrollServlet extends HttpServlet {
23
24     @Override
25     protected void doGet(HttpServletRequest request,
26                          HttpServletResponse response)
27         throws ServletException, IOException {
28
29         // Create ArrayList
30         List<Employee> list = new ArrayList<>();
31
32         // Employee 1
33         Employee e1 = new Employee();
34         e1.setEmpId("E001");
35         e1.setName("Ali Ahmad");
36         e1.setDepartment("IT");
```

```

37     e1.setBasicSalary(4500);
38
39     // Employee 2
40     Employee e2 = new Employee();
41     e2.setEmpId("E002");
42     e2.setName("Siti Nur");
43     e2.setDepartment("HR");
44     e2.setBasicSalary(2800);
45
46     // Employee 3
47     Employee e3 = new Employee();
48     e3.setEmpId("E003");
49     e3.setName("John Lee");
50     e3.setDepartment("Finance");
51     e3.setBasicSalary(3200);
52
53     // Employee 4
54     Employee e4 = new Employee();
55     e4.setEmpId("E004");
56     e4.setName("Muthu");
57     e4.setDepartment("Marketing");
58     e4.setBasicSalary(2500);
59
60     // Employee 5
61     Employee e5 = new Employee();
62     e5.setEmpId("E005");
63     e5.setName("Aina");
64     e5.setDepartment("Admin");
65     e5.setBasicSalary(5000);
66
67     // Add into list
68     list.add(e1);
69     list.add(e2);
70     list.add(e3);
71     list.add(e4);
72     list.add(e5);
73
74     // Share data with JSP
75     request.setAttribute("employeeList", list);
76
77     // Forward to JSP
78     RequestDispatcher rd =
79         request.getRequestDispatcher("payroll_view.jsp");
80
81     rd.forward(request, response);
82 }
83 }

```

...va displayList.jsp x jstl_database.jsp x Employee.java x PayrollServlet.java x payroll_view.jsp x

Source History

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35

```
<%--
Document    : payroll_view
Created on  : 12 May 2026, 6:37:41 pm
Author     : nursyaheera
--%>

<<@page contentType="text/html" pageEncoding="UTF-8"%>

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<!DOCTYPE html>
<html>
<head>
    <title>Employee Payroll Display System</title>

    <style>

        body{
            font-family: Arial;
        }

        table{
            border-collapse: collapse;
            width: 80%;
        }

        th, td{
            border: 1px solid black;
            padding: 10px;
            text-align: center;
        }

        th{
            background-color: lightblue;
        }
    
```

```

36
37     </style>
38
39 </head>
40
41 <body>
42
43     <h2>Employee Payroll List</h2>
44
45     <table>
46
47         <thead>
48
49             <tr>
50                 <th>No.</th>
51                 <th>Employee ID</th>
52                 <th>Name</th>
53                 <th>Department</th>
54                 <th>Basic Salary (RM) </th>
55                 <th>Status</th>
56             </tr>
57
58         </thead>
59
60         <tbody>
61
62             <c:forEach var="emp"
63                 items="{employeeList}"
64                 varStatus="status">
65
66                 <tr>
67
68                     <td>${status.count}</td>
69
70                     <td>${emp.empId}</td>
71
72                     <td>${emp.name}</td>
73
74                     <td>${emp.department}</td>
75
76                     <td>${emp.basicSalary}</td>
77
78                     <td>
79
80                         <c:choose>
81
82                             <c:when test="${emp.basicSalary >= 3000}">
83                                 Senior
84                             </c:when>
85
86                             <c:otherwise>
87                                 Junior
88                             </c:otherwise>
89
90                         </c:choose>
91
92                     </td>
93
94                 </tr>
95
96             </c:forEach>
97
98         </tbody>
99
100     </table>
101
102 </body>
103 </html>

```

Output:

Employee Payroll Display System

localhost:8081/Lab5_S76276/PayrollServlet

Employee Payroll List

No.	Employee ID	Name	Department	Basic Salary (RM)	Status
1	E001	Ali Ahmad	IT	4500.0	Senior
2	E002	Siti Nur	HR	2800.0	Junior
3	E003	John Lee	Finance	3200.0	Senior
4	E004	Muthu	Marketing	2500.0	Junior
5	E005	Aina	Admin	5000.0	Senior